

VegShift — Detailed Step-by-Step Guide (Steps 0–9)

This document explains Steps 0–9 of the VegShift pipeline in detail: purpose, inputs, key operations, sample input/output rows, validation checks, common caveats, and suggested next checks.

1. Step 0 — Master Index

Purpose: Create the canonical city × year backbone used to join all processed data.

Script: pipeline/step0_master_index.py

Command: ``python pipeline/step0_master_index.py``

Inputs: None (cities list and year range defined in script).

Outputs: ``data/processed/master_index.csv`` — 250 rows (10 cities × 25 years).

Key operations:

- Sample output row:** ``Delhi, 2000``
- Validation:** Verify file exists and row count equals 250. Check first/last city-year pairs.
- Caveats:** If the cities list is modified, downstream joins must be re-run.

2. Step 0b — Preprocess Raw Climate Data

Purpose: Standardize the daily climate dataset, compute derived fields (temp_mean, humidity), and normalize units.

Script: pipeline/step0b_preprocess_datasets.py

Command: ``python pipeline/step0b_preprocess_datasets.py``

Inputs: ``data/raw/climate/india_2000_2024_daily_weather.csv``

Outputs: ``data/processed/kaggle_climate.csv`` (daily rows with standardized columns).

Key operations:

- Rename source columns to ``temp_max``, ``temp_min``, ``rainfall``, ``wind_speed``.
- Compute ``temp_mean = (temp_max + temp_min)/2``.
- Convert wind speed units if needed.
- Estimate ``humidity`` from apparent temperature inversion when missing.
- Save with ``date`` parsed as datetime.
- Sample input row:** ``Delhi, 2000-01-01, 32.5, 21.1, 0.0``
- Sample output row:** ``Delhi, 2000-01-01, 32.5, 21.1, 26.8, 0.0, humidity: 74.2``
- Validation:** Ensure ``kaggle_climate.csv`` has daily rows from 2000–2024 and that ``temp_mean``, ``humidity`` columns exist with plausible ranges.
- Caveats:** Humidity is estimated — cross-check against station climatology for outliers.

3. Step 1 — Koppen-Geiger Classification

Purpose: Assign a Koppen climate zone to each city-year using annual and monthly aggregates.

Script: pipeline/step1_koppen_classification.py

Command: ``python pipeline/step1_koppen_classification.py``

Inputs: ``data/processed/kaggle_climate.csv``

Outputs: ``data/processed/koppen_annual.csv``, ``data/processed/zone_map.json``

Key operations:

- Aggregate daily data to monthly and annual summaries per city-year.
- Compute metrics: T_ann, P_ann, T_min (coldest month mean), T_max (hottest month mean), P_dry, P_wet, seasonal sums.
- Apply the simplified Koppen decision rules (aridity threshold P_{th} and temperature thresholds) to classify zones like ``BSh``, ``Aw``, ``Cwa``, etc.
- Encode zones into integer labels and persist ``zone_map.json``.
- Sample output row:** ``Delhi, 2005, BSh, 2``
- Validation:** Check that all 250 city-year rows have non-null ``koppen_zone``. Inspect zone frequency counts for plausibility.
- Caveats:** This implementation uses a simplified Koppen heuristic tuned for Indian climates; it may differ from authoritative gridded Koppen maps near transitional thresholds.

4. Step 1b — Transition Detection

Purpose: Detect persistent Koppen zone changes (climate transitions) per city.

Script: pipeline/step1b_transition_detection.py

Command: ``python pipeline/step1b_transition_detection.py``

Inputs: ``data/processed/koppen_annual.csv``

Outputs: ``data/output/transition_report.json``

Key operations:

- For each city, scan year-to-year zone changes.
- Require persistence of the new zone for PERSISTENCE (default 3) years to confirm a transition.
- Write transition events with ``city``, ``transition_year``, ``from_zone``, ``to_zone``, ``years_confirmed``.

****Sample output row:****

``{"city":"Lucknow","transition_year":2007,"from_zone":"Cwa","to_zone":"BSh"}``

- ****Validation:**** Confirm transitions are plausible (e.g., no single-year flickers). Inspect ``years_confirmed`` ≥ 3 for events.

- ****Caveats:**** The persistence requirement reduces false positives but may delay detection.

5. Step 2 — Climate Feature Aggregation

Purpose: Engineer crop-relevant annual features used by the model: monsoon onset, GDD accumulation, sowing-window miss, crop water deficit.

Script: `[pipeline/step2_climate_aggregate.py](pipeline/step2_climate_aggregate.py)`

Command: ``python pipeline/step2_climate_aggregate.py``

Inputs: ``data/processed/kaggle_climate.csv``, per-city CROP_CONFIG constants inside the script.

Outputs: ``data/processed/climate_annual.csv``

Key operations:

- Compute monsoon onset using IMD-inspired rule: first 5-day window after DOY 121 with ≥ 3 days ≥ 2.5 mm.

- Calculate Growing Degree Days (GDD) over the vegetative window (DOY 91–273) using crop-specific base temperature.

- Aggregate annual rainfall for the crop season and compute relative crop water deficit = $\max(0, (\text{water_req} - \text{season_rain})/\text{water_req})$.

- Compute ``sowing_window_miss`` as normalized delay relative to crop ``sow_doy``.

- Output per-city-year metrics: ``temp_mean``, ``rainfall_annual``, ``gdd_accumulation``, ``sowing_window_miss``, ``crop_water_deficit``, etc.

****Sample output row:**** ``Lucknow,2005,temp_mean:25.42,rainfall_annual:823.4,gdd_accumulation:3729.85,sowing_window_miss:1.0``

- ****Validation:**** Spot-check GDD values for monotonicity across warmer years; verify ``sowing_window_miss`` is within `[0,1]`.

- ****Caveats:**** Crop calendar constants (`sow_doy`, `water_req`) are expert choices inside ``CROP_CONFIG`` and impact the derived features.

6. Step 3 — Groundwater Aggregation

Purpose: Aggregate CGWB well observations around each city to estimate pre- and post-monsoon groundwater depth and recharge efficiency.

Script: `[pipeline/step3_groundwater_aggregate.py](pipeline/step3_groundwater_aggregate.py)`

Command: ``python pipeline/step3_groundwater_aggregate.py``

Inputs: ``data/raw/cgwb/CGWB_India_quality_controlled_GWLs_ref_sy_2000_2022.csv``, city coordinates in script.

Outputs: ``data/processed/groundwater_annual.csv``

Key operations:

- Compute haversine distance from city to each well and select wells within RADIUS_KM (default 50 km).
- If none present (Jaipur case), fallback to the 5 nearest wells and flag ``gw_imputed=2``.
- For each year, take the median of May (pre-monsoon) and Nov (post-monsoon) depths across nearby wells.

- Compute ``depletion_rate`` (year-to-year diff of pre-monsoon depth) and ``recharge_efficiency`` = $\text{depth_recovery} / \text{rainfall_annual}$.

- Backfill early years' `depletion_rate` with mean from 2005–2007 if missing.

****Sample output row:**** ``Ahmedabad,2005,pre_monsoon_depth_mbgl:7.98,post_monsoon_depth_mbgl:2.075,depletion_rate:-1.17``

- ****Validation:**** Confirm ``n_wells`` > 0 for most cities, check ``gw_imputed`` flags, and ensure ``recharge_efficiency`` within `[0,1]` after clipping.

- ****Caveats:**** Well coverage is sparse for some regions; nearest-well fallbacks should be interpreted as lower-fidelity estimates.

7. Step 4 — FAO GAEZ Extraction

Purpose: Extract baseline crop suitability class for each city's coordinate from FAO GAEZ GeoTIFFs and map to ECOCROP thresholds.

Script: pipeline/step4_gaez_extract.py

Command: ``python pipeline/step4_gaez_extract.py``

Inputs: ``data/raw/gaez/*.tif`` (per-crop suitability rasters), city coordinates.

Outputs: ``data/processed/gaez_baseline.csv``

Key operations:

- Open the relevant GeoTIFF for each city and read the raster value at the city's lon/lat.
- Clip values > 7 to 7 to keep rainfed suitability scale; irrigated-potential codes (8–10) are mapped down.
- Attach ECOCROP thresholds (``gdd_min``, ``water_req``) from script lookup for model reference.
`**Sample output row:**`Lucknow, gaez_baseline_class:7 (clipped from 8), gdd_min:2500, water_req:1500``
- **Validation:** Ensure raster read succeeds and ``gaez_baseline_class`` is between 1–7 after clipping.
- **Caveats:** Single-point sampling at city coordinates may not reflect intra-city heterogeneity.

8. Step 5 — Join + Features + CVLE Labels

Purpose: Merge climate, groundwater, GAEZ, and Koppen data onto the master index; compute compound features and label Crop Viability Loss Events (CVLE).

Script: pipeline/step5_join_and_features.py

Command: ``python pipeline/step5_join_and_features.py``

Inputs: ``data/processed/master_index.csv``, ``climate_annual.csv``, ``groundwater_annual.csv``, ``gaez_baseline.csv``, ``koppen_annual.csv``

Outputs: ``data/processed/vegshift_master.csv`` (master table with engineered features and ``cvle_label``).

Key operations:

- Three-way join (master $\leftarrow$ climate $\leftarrow$ groundwater $\leftarrow$ gaez) and koppen merge per year.
- Per-city group imputation for missing numeric features using group means.
- Create compound features:
 - ``dual_deficit`` = atmospheric crop_water_deficit > 0.4 AND ``recharge_efficiency`` < 0.30
 - ``gdd_adequate`` = ``gdd_accumulation`` $\geq$ ``gdd_min``
 - CVLE labeling rule: a row is labeled ``cvle_label=1`` when
 - ``dual_deficit`` is true for 2+ consecutive years AND
 - At least 2 of the 3 thresholds are breached in the second year: ``sowing_window_miss`` > 0.6 , ``crop_water_deficit`` > 0.4 , ``gdd_adequate`` $== 0$
- `**Sample output row:**`Lucknow,2005, dual_deficit:1, gdd_adequate:1, cvle_label:1``
- **Validation:** Check that ``vegshift_master.csv`` has 250 rows and the ``cvle_label`` column exists. Inspect per-city CVLE counts for plausibility.
- **Caveats:** The CVLE rule is conservative (requires consecutive dual_deficit + multiple threshold breaches); adjust thresholds for sensitivity testing.

9. Step 6 — TFT Training

Purpose: Train a Temporal Fusion Transformer to predict CVLE probability using a 5-year lookback.

Script: pipeline/step6_tft_train.py

Command: ``python pipeline/step6_tft_train.py``

Inputs: ``data/processed/vegshift_master.csv``

Outputs: ``models/tft/vegshift-tft-best*.ckpt`` (best checkpoint saved by ModelCheckpoint)

Key operations:

- Prepare ``TimeSeriesDataSet`` with: ``encoder_length=5``, ``prediction_length=1``.
- Specify static and time-varying features, include ``koppen_zone_enc`` as a time-varying categorical.
- Train TFT with QuantileLoss and early stopping; save best checkpoint.
- **Validation:** Confirm a checkpoint is produced and monitor training/validation loss curves in ``lightning_logs/``.
- **Caveats:** TFT training can be sensitive to hyperparameters and class imbalance; consider class weighting, oversampling, or alternative loss formulations.

10. Step 7 — TFT Predict + Attention

Purpose: Load the best TFT checkpoint, generate predicted CVLE probabilities, and extract attention weights for interpretability.

Script: pipeline/step7_tft_predict.py

Command: `python pipeline/step7\_tft\_predict.py`

Inputs: `data/processed/vegshift\_master.csv`, `models/tft/vegshift-tft-best\*.ckpt`

Outputs: `data/output/tft\_predictions.csv`, `data/output/tft\_attention\_weights.json`,
`data/output/crop\_viability\_events.json`

Key operations:

- Reconstruct the TimeSeriesDataSet used for validation.
- Call `predict()` on the validation loader and aggregate predicted quantiles to median probability for CVLE.
- Use `interpret\_output()` to extract per-sample encoder attention weights and average them per city.
- Export CVLE events (rows where `cvle\_label==1`) to JSON for downstream dashboard use.
- **Sample prediction row:** `Delhi,2021,0.0096`
- **Sample attention:** `Lucknow: [0.20,0.21,0.15,0.22,0.23]`
- **Validation:** Check predictions CSV has one row per city-year, attention weights sum ≈ 1 for each encoder window.
- **Caveats:** If the model was trained without proper class balancing, predicted probabilities may be poorly calibrated.

11. Step 8 — Baselines (RF, LR, LSTM)

Purpose: Train simpler baselines (Random Forest, Logistic Regression, LSTM) for comparison to the TFT.

Script: pipeline/step8_baselines.py

Command: `python pipeline/step8\_baselines.py`

Inputs: `data/processed/vegshift\_master.csv`

Outputs: `models/baselines/rf\_baseline.pkl`, `models/baselines/lr\_baseline.pkl`,
`models/baselines/lstm\_baseline.pt`, `models/baselines/scaler.pkl`, `data/output/baseline\_metrics.json`

Key operations:

- Scale features with `StandardScaler`.
- Train RF and LR on a time-based split (train ≤ 2018 , test ≥ 2022) and report classification metrics (precision/recall/F1, AUC).
- Build an LSTM sequence classifier using sequences of length 5 as a temporal baseline.
- **Sample metrics (observed):** `RF AUC=1.0, LR AUC=1.0, LSTM AUC $\approx$ 0.846` — note these can be misleading due to test-set class imbalance.
- **Validation:** Inspect `data/output/baseline\_metrics.json` and model files under `models/baselines/`.
- **Caveats:** Perfect AUCs often indicate a tiny or unrepresentative test set; prefer time-series cross-validation or aggregated test windows.

12. Step 9 — SHAP Explainability

Purpose: Compute SHAP values for the RF baseline to obtain global and per-city feature importances.

Script: pipeline/step9_shap_explainability.py

Command: `python pipeline/step9\_shap\_explainability.py`

Inputs: `data/processed/vegshift\_master.csv`, `models/baselines/rf\_baseline.pkl`,
`models/baselines/scaler.pkl`

Outputs: `data/output/shap\_explanation.json`

Key operations:

- Apply `scaler` to features, use `shap.TreeExplainer` on RF, compute per-sample SHAP contributions.
- Aggregate mean absolute SHAP for global importance and mean per-city importances (top-5 features).
- **Sample global top features:** `recharge\_efficiency`, `crop\_water\_deficit`, `gdd\_accumulation`.
- **Validation:** Check that shap_explanation.json contains `global\_importance` and `city\_importance` entries.
- **Caveats:** SHAP values reflect the RF model's reasoning; if RF overfits (perfect AUC), SHAP will reflect overfit patterns.

Recommended immediate checks after running Steps 0–9

- Confirm the presence and row counts of these core files:
- data/processed/master_index.csv
- data/processed/kaggle_climate.csv

- data/processed/vegshift_master.csv
- `models/tft/vegshift-tft-best\*.ckpt`
- `models/baselines/rf\_baseline.pkl`
- `data/output/tft\_predictions.csv`, `data/output/shap\_explanation.json`, `data/output/transition\_report.json`

Notes on evaluation and robustness

- Use time-series cross-validation or rolling-window validation rather than a single train/test split to avoid unstable AUCs.
- Prefer PR-AUC and class-wise precision/recall when positive events are rare.
- Calibrate probabilities (Platt/Isotonic) before choosing operational CVLE thresholds.
- Run sensitivity analyses on CVLE thresholds (sowing_window_miss, crop_water_deficit, gdd) and persistence requirement to quantify detection stability.

If you want this saved as a prettier PDF

- I can regenerate the pretty PDF to include this expanded text. Command to run locally (from repo root):

```
& "./.venv/Scripts/Activate.ps1"
python scripts/generate_vegshift_report_pretty.py
```